



CSI247: Data Structures

Lecture 14 – LinkedList implementation

Department of Computer Science

B. Gopolang (247-275)

Previous lesson

- Java API Collections
 - A collection of Interfaces and Classes for handling collections
 - Examples
 - Interfaces: List, Queue, Set
 - Classes: They implement interfaces
 - ArrayList, LinkedList, PriorityQueue, Stack, etc
 - Used Java's API to understand these classes
 - ArrayList (A dynamic array)
 - LinkedList (Nodes linked together via pointers)
 - Stack (LIFO)
 - Queue (FIFO)

Today ...

Implementing LinkedList class

Lecture objectives

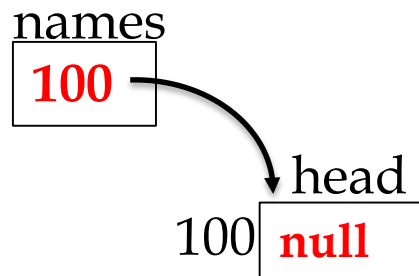
- By the end of this lecture, you must be able to:
 - Understand and implement an inner class
 - Understand and implement LinkedList constructor
 - Understand and implement the rest of the LinkedList class methods
 - Test the LinkedList class

1. Review of a LinkedList

- Linked list:
 - Object that stores a linear collection (nodes)
 - Each element is a separate node (object)
 - Nodes are stored in a non-contiguous manner
 - Unlike in an array
 - Has a **head** (1st node) and **tail** (last node)
 - First node reached via a reference to the linked list object
 - Last node's reference is always null (end of list)

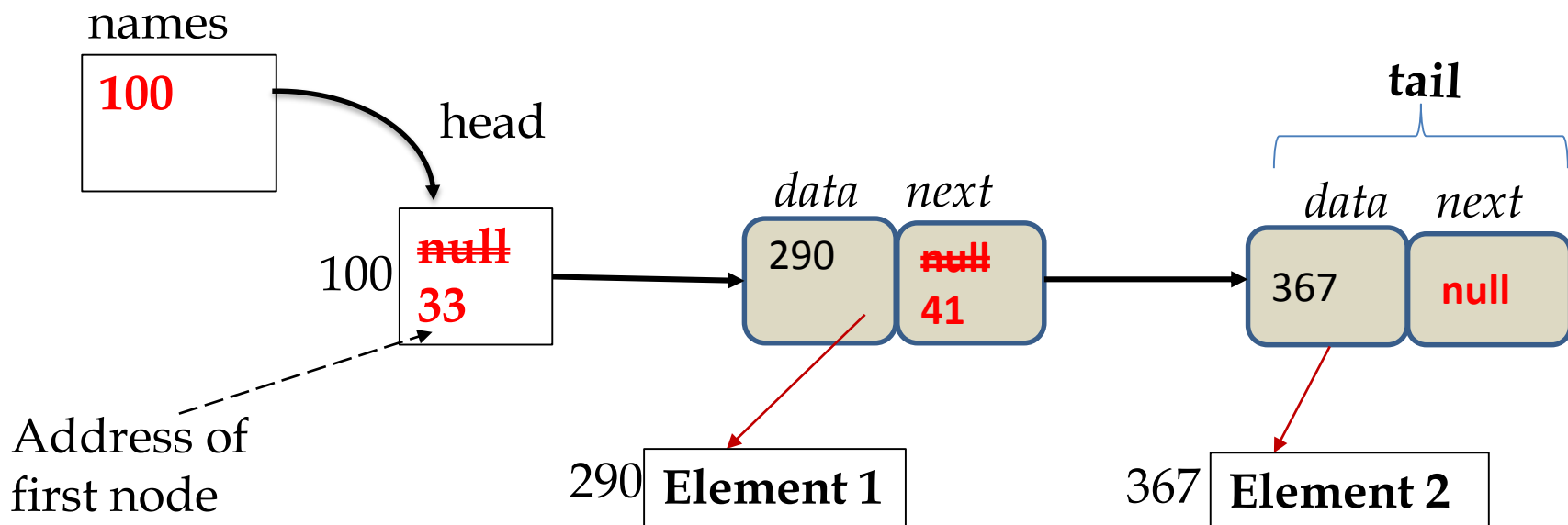
- Entry point into a linked list is called the **head**
 - Reference to first node (next slide)
 - **null** if LinkedList is empty

Empty linked list



- A recursive data structure
 - Either **empty** (head == null)
 - or there is a **node** that contains data item and a reference to the **next node**

Linked list with two nodes



● Nodes:

- Each node stores:
 - Data
 - Reference/address to next node (pointer)
- Each node has:
 - Only one node preceding it (except possibly first node)
 - Only one node succeeding it (except possibly last node)

Rem:

- Nodes may not be stored contiguously in memory unlike in an array

- **Types of Linked lists:**

- a) Singly linked list

- Each node has 1 link: next node only

- b) Doubly linked list

- Each node has 2 links: previous and next node

- c) Circular linked list

- Last node has a link to first node

● Accessing LinkedList elements

- Array:
 - We use **arr[]** to access each item (direct access)
- ArrayList
 - **get()** – still direct access
- LinkedList:
 - [] does not work!!
 - **Solution:**
 - Keep track of the **head** – reference to 1st node
 - Then we follow the links to access any element
 - Sequential access

- **Benefits of a linked list?**

a) A dynamic structure

- Grows/shrinks whenever necessary

b) Efficient compared to arrays & Arraylist

- No need to shift items for insertions and removals

- **Draw back?**

- No direct access to elements

2. Non-generic implementation of Singly LinkedList

- Note:
 - Linked List is an **object**
 - Each node is also a separate **object**
 - So, we need to implement 2 classes
- We need:
 - a) LinkedList class (for the LinkedList object)
 - b) Node class (for Node objects)
 - As a separate class
 - Or an inner class
 - c) Then a Tester class to test our implementation

Using an inner class

- Implementing one class inside another class
- AKA nested classes
- Syntax

```
public class OuterClass{  
    // ... outer class functionality  
    private class InnerClass {  
        // ... inner class functionality  
    }  
    // ... more outer class functionality  
}
```

- **Why nested classes**
 - To group classes belonging together
 - Results into a more readable and maintainable code
- **How to access inner class?**
 - Create outer class object first
 - Then create inner class object inside outer class
 - Will become clearer when we write our LinkedList class

Back to implementation

i) Inner class (Node)

```
private class Node{  
    int data;      // node's data part  
    Node next;    // a link to another node  
    Node (int data) { // another Constructor  
        this.data = data;  
        this.next = null; // assumption: it will be the last Node in the list  
    }  
    Node (int data, Node newNode) { // Constructor  
        this.data = data;  
        this.next = newNode; //  
    }  
}
```

Initialising instance variables:
data: receives the data parameter
next: initially null - no node after this one

Used when we need to create a new node

- How will we know number of elements?
 - Need a counter variable, we will call it **nodes**
 - Where do we store it?
 - LinkedList or Node?
 - No need for every node to store size
 - So, keep it in LinkedList class
 - What else should LinkedList keep track of?
 - Head node
 - Tail node: traverse the list to reach it (if no direct link to it)

Initial look at our implementation

ii) Outer class (LinkedList)

```
public class LinkedList {
```

Private class Node goes here!

// inner class

// instance variable

```
public Node head; // to keep track of 1st node
```

```
public Node tail; // to keep track of last node
```

```
public int nodes; // to keep track of number of nodes in the link
```

// Constructor

// Rest of methods - to be added as we build the class

```
}
```

b) Implementing LinkedList methods

i. Constructor

- Purpose?
 - To create of LinkedList object
 - Rem: it initializes instance variables!

```
public LinkedList() {  
    head = null;           // Initially, no node pointed to by head  
    tail = null;           // no tail node  
    nodes = 0;              // list is initially empty  
}
```

ii. size()

- Purpose?
 - Specifies no. of nodes in the LinkedList
- Method Signature?
 - **Return type:** int, number of nodes in the list
 - **Parameters:** None

```
public int size() {
```

```
    return nodes; // WHAT IF node is not defined?
```

```
    // Soln: traverse the LinkedList & count nodes individually!
```

```
}
```

iii. isEmpty()

- Purpose?
 - To determine if the LinkedList has nodes
 - How do we know when it is empty?
 - Rem: **head** variable?

```
public boolean isEmpty() {  
    return (head == null) ;  
}
```

- Testing our incomplete LinkedList class
- Sample code fragment:

// create an object

```
LinkedList myList = new LinkedList(); // non-generic!
```

// checking if the linked list if empty

```
System.out.println("List empty? : " + myList.isEmpty());
```

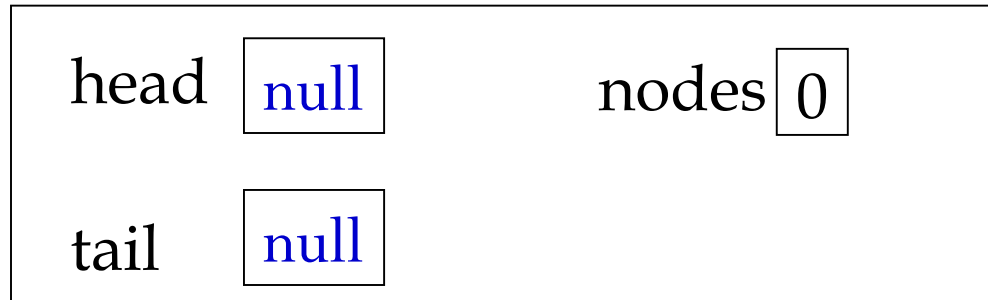
// displaying number of nodes in the list

```
System.out.println("List has " + myList.size() + " elements");
```

Exercise

- We will implement LinkedList class incrementally over 2 or 3 lectures.
- Your task:
 - Type code segments provided thus far in the handout into one class file and compile it. Fix errors, if any.
 - Create a Tester class and include the following functionality in it:
 - Create a LinkedList object using the incomplete class
 - Display size of the list
 - Display message “Empty list!” when the list is empty, otherwise display “List not empty”;

- So far:
 - We can create a LinkedList
 - Then get its size & check if it is empty!
- Current memory diagram situation :



- More methods needed for the rest of the functionality

Rest of the methods...

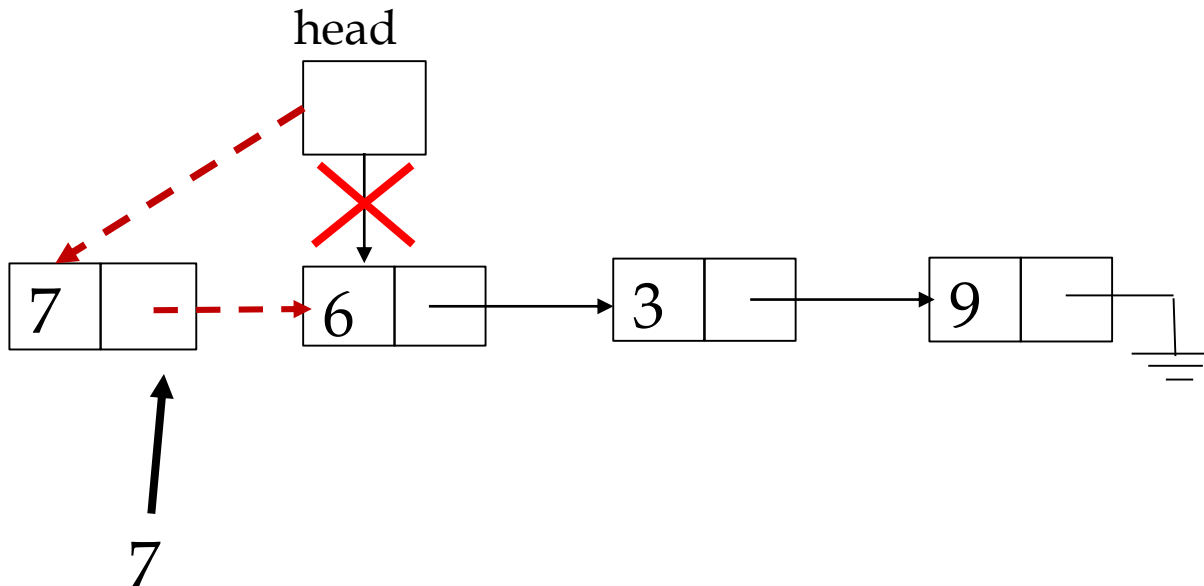
iv. Add a node

- Purpose?
 - To insert elements into the LinkedList object
- Several options

➤ Add at the front: **addFront()**

- **Steps to follow:**
 - Create new node, store **data** & **next** (initially **null**)
 - Adjust pointers accordingly
 - Change head's pointer: must point to new node
 - To make new node the head node

- E.g. Let us add 7 at the beginning of this LinkedList:
- We MUST replace current head with new node



```
public void addFront(int data){  
    // create node  
    Node n = new Node(data);  
    // where do we add it?  
    if (isEmpty()) { // list empty?  
        head = n;  
        tail = n;  
    } // n: both 1st and last node!
```

```
else {  
    Node tmp = head; // preserve  
    address of the current head  
    head = n; // n is new head  
    n.next = tmp; // link current  
    head node with previous head node  
}  
nodes++; // nodes increases by 1  
}
```

- Time complexity?

– $O(1)$?

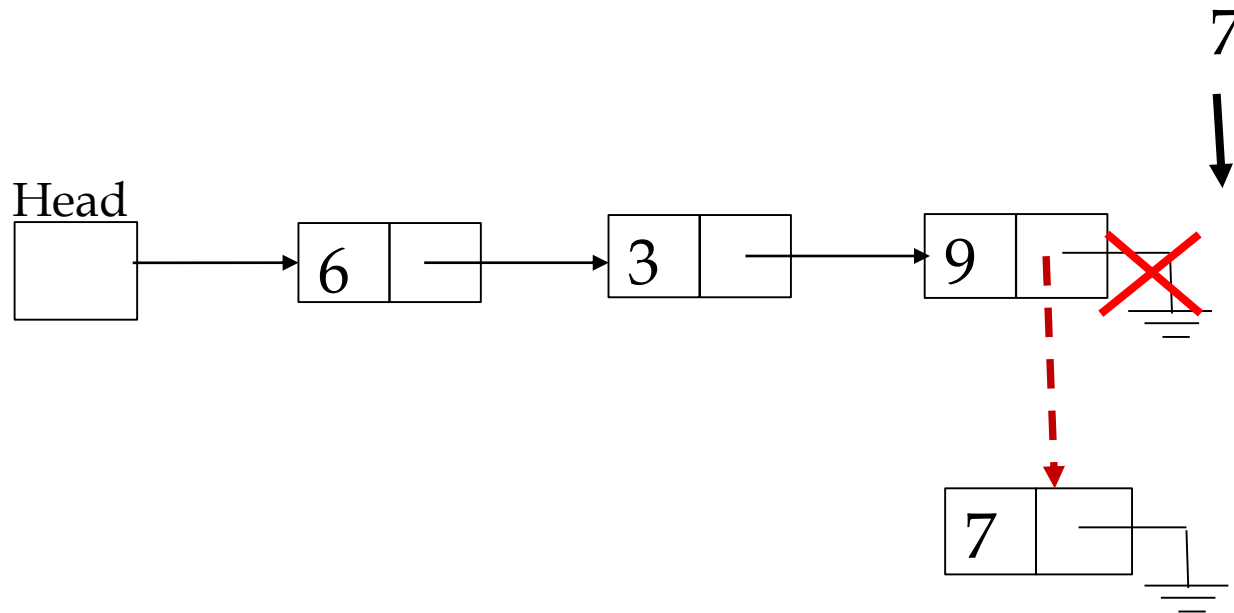
➤ Add at the end: **addLast()**

- New node MUST become the tail node

Steps to follow:

- Create new node & store data
- Traverse list to last node (end of list)
 - Rem: variable **tail** is used to keep track of this
- Change next of last node to point to new node (new node become the new last node)

- E.g. Adding 7 at the end:



```
public void addLast(int data){  
    // create new node  
    Node n = new Node(data);  
    // Add new node to list  
    if (isEmpty()) { // empty list  
        head = n; // it's 1st node  
        tail = n; // and last node  
    }
```

```
else {  
    // node becomes new tail  
    tail.next = n; // create link  
    tail = n; // it's new tail  
}  
nodes++; // nodes up by 1  
}
```

● Time complexity?

- For current implementation:
 - $O(1)$?
 - We have direct access to tail node
- What if head is the only instance variable?
 - Traverse the entire list to reach last node
 - Means visiting n nodes (where $n = \text{no. of nodes}$)
 - Hence, $O(n)$

➤ At a specified position: add (int i)

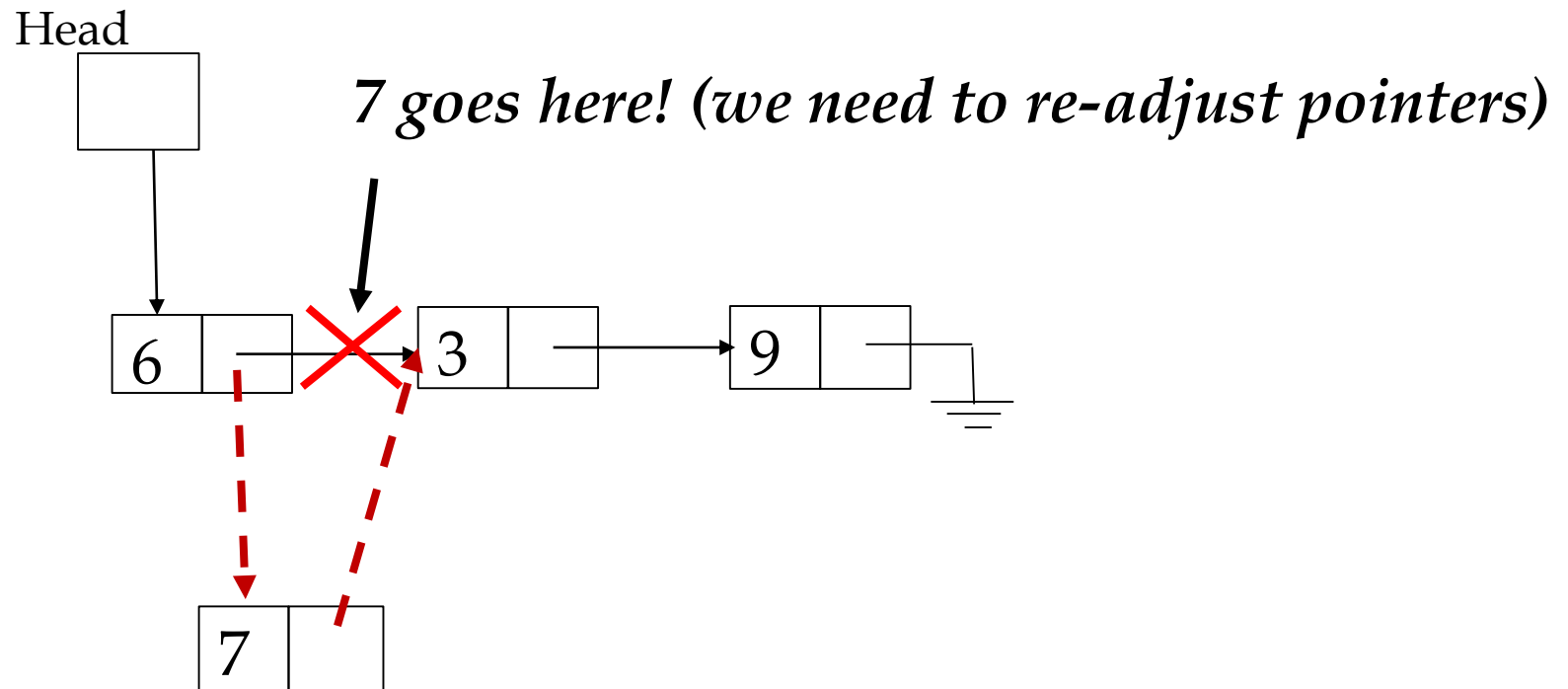
- **Steps to follow:**

- Create new node
- Traverse list until node before specified position
- Let next of the node before index i point to new node
- Let new node's next point to right neighbour

- Take home exercise:

- Implement this method!
- Time complexity? Best case, worst case?

- E.g. Adding as second node:



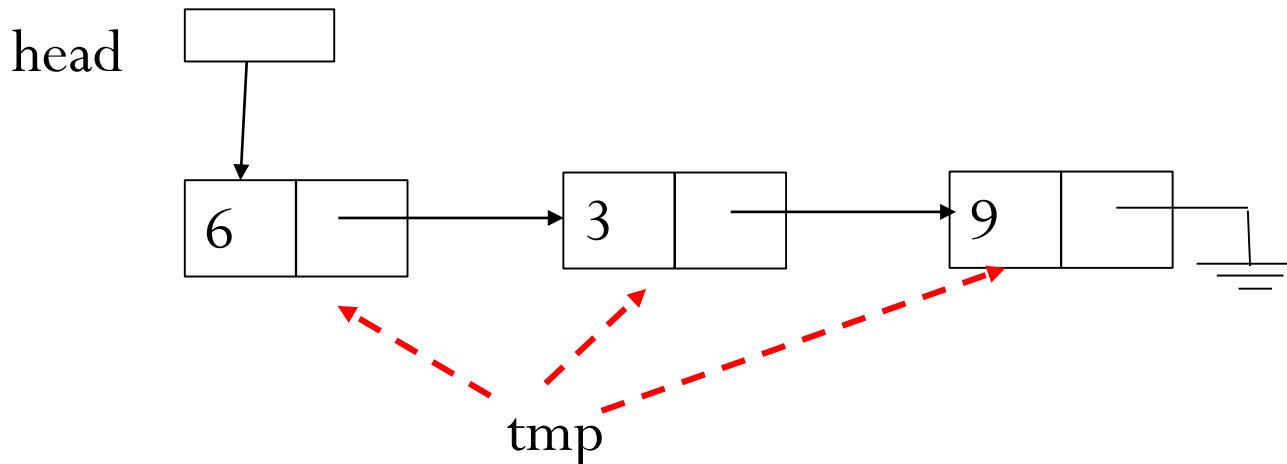
v. contains()

● Purpose?

- To check if the LinkedList object has a specified node
- Requires traversing the list (see next slide)
- **Steps** to follow:
 - Make head the current node
 - Traverse the list until last node
 - During each iteration:
 - Compare node's data with the search key
 - Returns **true** when they match, **false** otherwise

● Traversing the list?

- Means going through the list, from 1st item until the last item
- E.g.: Searching for an element in a list
- Start from head until
 - **stop** : node found (rem: efficiency is crucial!)
 - **exhaust the list**: node NOT found



```
public boolean contains(int data){  
    Node tmp = head;      // start searching from the head  
    while (tmp != null) {  // stop upon reaching null  
        if (tmp.data == data) // they are similar  
            return true;     // found, stop searching!  
        tmp = tmp.next;     // move to next node  
    }  
    return false;          // not found  
}
```

- Time complexity

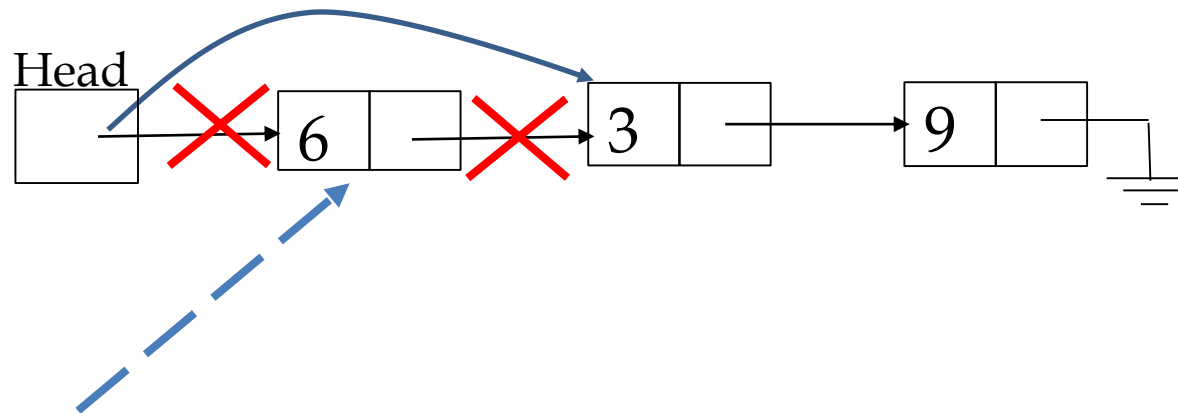
- Best case: $O(1)$, worst case: $O(n)$?

vii. Remove nodes

- Purpose?
 - To remove elements from the LinkedList object
- Several options
 - From the front (beginning): **removeFront()**
 - **Steps to follow:**
 - Check if list is not empty
 - Change head to point to second node

Removing from front:

- Suppose we have this Linkedlist structure:



- Node is isolated from the rest
- Garbage collector will reclaim its memory space

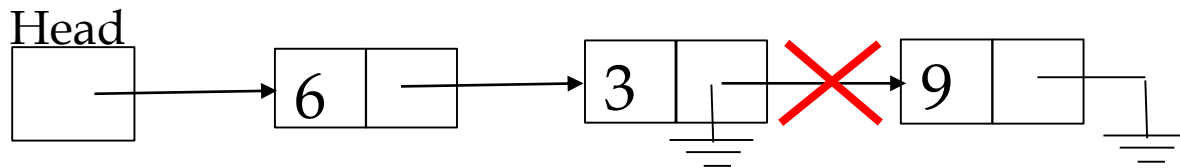
```
public void removeFront() {  
    if (isEmpty())  
        System.out.println("List is empty");  
    else {  
        head = head.next; // promote next node as head  
        nodes--;  
    }  
}
```

- Time complexity
 - $O(1)$?

➤ At the tail(end): **removeLast()**

- **Steps to follow:**

- Traverse list until second last node
- Replace its next with null
 - This will isolate old tail from rest of the list



```
public void removeLast() {  
    if (isEmpty())  
        System.out.println("List is empty");  
    Node cur = head; // current node is head  
    while (cur.next.next != null) { // Traverse list to locate node before last  
        cur = cur.next;  
    }  
    cur.next = null; // delete tail  
    nodes--;  
}
```

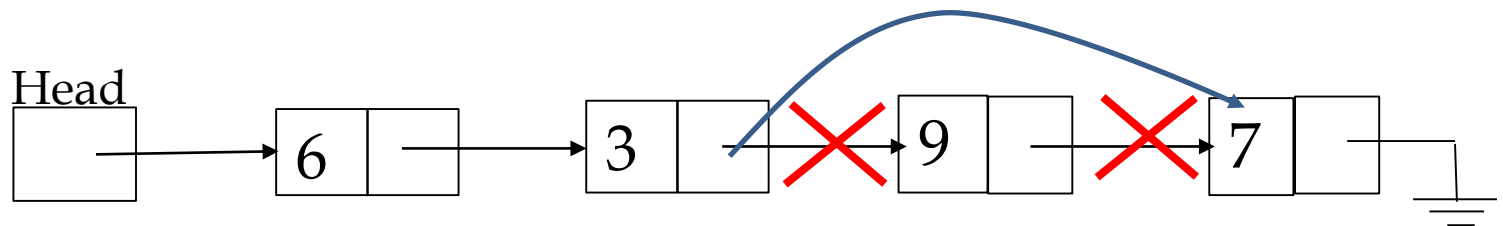
- Time complexity

- $O(?)$

➤ At a specified position: **remove(int i)**

- Steps to follow:

- Traverse list until node before specified position
- Change its pointer to bypass the node from the list



- Take home exercise:
 - Implement this method!
 - Time complexity?

Exercise

- Your task:
 - Add code segments from the previous file and compile it. Fix errors, if any.
 - Create a Tester class and include the following functionality in it:
 - Create a LinkedList object and add 6 nodes.
 - Display the nodes, together with the number of nodes
 - Remove 1st node and print remaining nodes
 - Add 2 nodes in front, display all nodes

- Display “Found it” if the list has 120, otherwise display “Better luck next time!”
 - Any suggestions on how to improve any of the methods?
 - Implement the remaining 2 methods and test them
-
- Create a generic LinkedList class called GenLinkedList and test it with String, Integer, Dog values.

Summary

- LinkedList Implementation structure

LinkedList class

Node class

Node Instance variable(s)

Node Constructors

LinkedList instance variable(s)

LinkedList Constructor(s)

Rest of LinkedList methods

Tester class

main()

Create LinkedList

Use methods defined and
tested in LinkedList class

6. Summary

- Implementation of a linkedList
- Various methods
 - Constructor
 - size()
 - isEmpty()
 - Addition method(s)
 - Deletion method(s)
 - contains()

Next lesson ...

Implementation
of a Stack ADT

Q & A



Thank you